CS 3244

Machine Learning

Week 5 – Kernelization

How can machines learn?

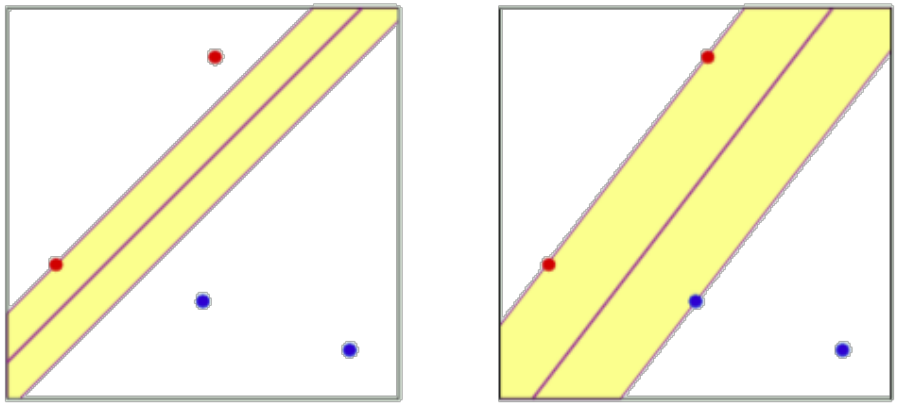
LFD, dynamic e-Chapter 8

Kan Min-Yen

Background Photo credits: Rafiq Mirza, Luan Ahn, Rosmarie Voegtli @ Flickr

Summary

Fat margins are good!



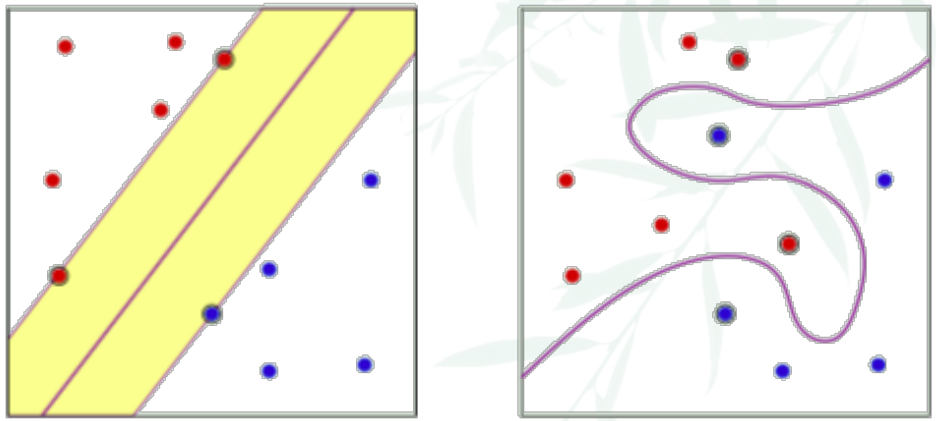
Maximizing the margin:

$$\mathcal{L}(\boldsymbol{\alpha}) = \sum_{n=1}^{N} \alpha_n - \frac{1}{2} \sum_{n=1}^{N} \sum_{m=1}^{N} y_n y_m \alpha_n \alpha_m \, \boldsymbol{x}_n^T \boldsymbol{x}_m$$

Solve using Quadratic Programming

Support Vectors:

$\boldsymbol{x}_n$ or $\boldsymbol{z}_n$ with Lagrange $\alpha_n > 0$



Soft margin SVM

Minimize $\frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} + C\sum_{n=1}^{N}\xi_n$

$\xi_n \equiv$ slack per point; trade off parameter C (slack vs. margin)

What do we need from the $\mathcal{Z}$ space?

$$\mathcal{L}(\boldsymbol{\alpha}) = \sum_{n=1}^{N} \alpha_n + 0 - \frac{1}{2} \sum_{n=1}^{N} \sum_{m=1}^{N} y_n y_m \alpha_n \alpha_m \, \boldsymbol{z}_n^T \boldsymbol{z}_m$$

Constraints: $\alpha_n \geq 0$ for $n = 1, \ldots, N$ and $\sum_{n=1}^{N} \alpha_n y_n = 0$

To classify need to compute:

$g(\boldsymbol{x}) = \text{sign}(\boldsymbol{w}^T \boldsymbol{z} + b)$ $\rightarrow$ need $\boldsymbol{z}_n^T \boldsymbol{z}_m$

where $\boldsymbol{w} = \sum_{z_n \in SV} \alpha_n y_n \boldsymbol{z}_n$

and b: $y_m(w^T z_m + b) = 1$ $\rightarrow$ need $\boldsymbol{z}_n^T \boldsymbol{z}_m$

Generalized inner product

Given two points $\boldsymbol{x}$ and $\boldsymbol{x}' \in \mathcal{X}$, we just need $\boldsymbol{z}^T\boldsymbol{z}'$

Let $\boldsymbol{z}^T\boldsymbol{z}' = K(\boldsymbol{x}, \boldsymbol{x}')$ (the kernel) "inner product" of $\boldsymbol{x}$ & $\boldsymbol{x}'$

Example: 2nd order Polynomial <u>Transform</u>:

$$\boldsymbol{x} = \underbrace{< x_1, x_2 >}_{d \text{ is } 2}$$

Components of x. Not bolded.

$$\mathbf{z} = \Phi(\boldsymbol{x}) = (1, x_1, x_2, \sqrt{2}x_1^2, \sqrt{2}x_2^2, \sqrt{2}x_1x_2)$$

Ignore scaling C

$$K(\boldsymbol{x}, \boldsymbol{x}') = \boldsymbol{z}^T\boldsymbol{z}' = 1 + x_1x_1' + x_2x_2' + 2x_1^2x_1'^2 + 2x_1^2x_1'^2 + 2x_1x_1'x_2x_2'$$

The trick

Can we compute $K(\boldsymbol{x}, \boldsymbol{x}')$ without transforming $\boldsymbol{x}$ and $\boldsymbol{x}'$?

Example: Polynomial Kernel

Consider $K(\boldsymbol{x}, \boldsymbol{x}') = (1 + \boldsymbol{x}^T \boldsymbol{x}')^2$ for same 2-dimensional $\boldsymbol{x}$

$$= (1 + x_1 x_1' + x_2 x_2')^2$$

$$= 1 + x_1^2 x_1'^2 + x_2^2 x_2'^2 + 2x_1 x_1' + 2x_2 x_2' + 2x_1 x_1' x_2 x_2'$$

This is an inner product!

$$\boldsymbol{x} = \left(1, x_1^2, x_2^2, \sqrt{2}x_1, \sqrt{2}x_2, \sqrt{2}x_1 x_2\right)$$

$$\boldsymbol{x}' = \left(1, x_1'^2, x_2'^2, \sqrt{2}x_1', \sqrt{2}x_2', \sqrt{2}x_1' x_2'\right)$$

The polynomial kernel

$\mathcal{X} = \mathbb{R}^d$ and $\Phi: \mathcal{X} \to \mathcal{Z}$ is a polynomial of order Q

Compare for $d = 10$ and $Q = 100$: $(1 + \boldsymbol{x}^T \boldsymbol{x}')^Q$

1. As a <u>transform</u>:
   - Calculate $\Phi(\boldsymbol{x})^T \Phi(\boldsymbol{x}')$
   - Take $\boldsymbol{x}$ and $\boldsymbol{x}'$, explicitly expand vectors through $\Phi(\boldsymbol{x})$:

     $\Phi(\boldsymbol{x}) = (1, x_1, x_2, \dots, x_d, \dots, x_1 x_2, \dots, x_9 x_{10}, \dots, x_1^{100}, x_2^{100}, \dots, x_{10}^{100})$

     $\Phi(\boldsymbol{x}') = (1, x_1', x_2', \dots, x_d', \dots, x_1' x_2', \dots, x_9' x_{10}', \dots, x'^{100}_1, x'^{100}_2, \dots, x'^{100}_{10})$

   Takes O(Q^2) computations.

2. As a <u>function</u>:
   - Compute $K(\boldsymbol{x}, \boldsymbol{x}') = (1 + \sum_d x_d x_d')^Q$ directly:

   $(1 + x_1 x_1' + x_2 x_2' + \cdots + x_d x_d')^Q$ by summing d terms to a scalar, then take to Q^{th} power. Takes O(d) computations.

We only need $\mathcal{Z}$ to exist

If $K(\boldsymbol{x}, \boldsymbol{x}')$ is an inner product in <u>some</u> space $\mathcal{Z}$, we are good

Example (Radial Basis Function Kernel):

$K(\boldsymbol{x}, \boldsymbol{x}') = \exp(-\gamma \|\boldsymbol{x} - \boldsymbol{x}'\|^2)$, where $\gamma = \frac{1}{2\sigma^2}$

Infinite-dimensional $\mathcal{Z}$: take the simple case

$$\begin{aligned} K(\boldsymbol{x}, \boldsymbol{x}') &= \exp(-(\boldsymbol{x} - \boldsymbol{x}')^2) \\ &= \exp(-\boldsymbol{x}^2) \exp(-\boldsymbol{x}'^2) \underbrace{\sum_{k=0}^{\infty} \frac{2^k (x)^k (x')^k}{k!}}_{\exp(2\boldsymbol{x}\boldsymbol{x}')} \end{aligned}$$

Get the benefit of an infinite $\mathcal{Z}$,

but still pay O(d) computations.

Gaussian Kernel

(aka Radial Basis Function Kernel): $K(\boldsymbol{x}, \boldsymbol{x}') = e^{-\gamma\|\boldsymbol{x}-\boldsymbol{x}'\|^2}$

Example: Gaussian Kernel with $d = 1$

$$\Phi(\boldsymbol{x}) = \begin{bmatrix} e^{-x^2}\sqrt{\frac{2^0}{0!}} \\ e^{-x^2}\sqrt{\frac{2^1}{1!}}x \\ e^{-x^2}\sqrt{\frac{2^2}{2!}}x^2 \\ e^{-x^2}\sqrt{\frac{2^3}{3!}}x^3 \\ \vdots \end{bmatrix}$$

(infinite dimensional Φ)

$$K(\boldsymbol{x}, \boldsymbol{x}') = \Phi(\boldsymbol{x})^T\Phi(\boldsymbol{x}') = \begin{bmatrix} e^{-x^2}\sqrt{\frac{2^0}{0!}} \\ e^{-x^2}\sqrt{\frac{2^1}{1!}}x \\ e^{-x^2}\sqrt{\frac{2^2}{2!}}x^2 \\ e^{-x^2}\sqrt{\frac{2^3}{3!}}x^3 \\ e^{-x^2}\sqrt{\frac{2^4}{4!}}x^4 \\ \vdots \end{bmatrix} \cdot \begin{bmatrix} e^{-x'^2}\sqrt{\frac{2^0}{0!}} \\ e^{-x'^2}\sqrt{\frac{2^1}{1!}}x' \\ e^{-x'^2}\sqrt{\frac{2^2}{2!}}x'^2 \\ e^{-x'^2}\sqrt{\frac{2^3}{3!}}x'^3 \\ e^{-x'^2}\sqrt{\frac{2^4}{4!}}x'^4 \\ \vdots \end{bmatrix}$$

$$= e^{-\boldsymbol{x}^2}e^{-\boldsymbol{x}'^2}\sum_{i=0}^{\infty}\frac{(2\boldsymbol{x}\boldsymbol{x}')^i}{i!}$$

$$= e^{-(\boldsymbol{x}-\boldsymbol{x}')^2}$$

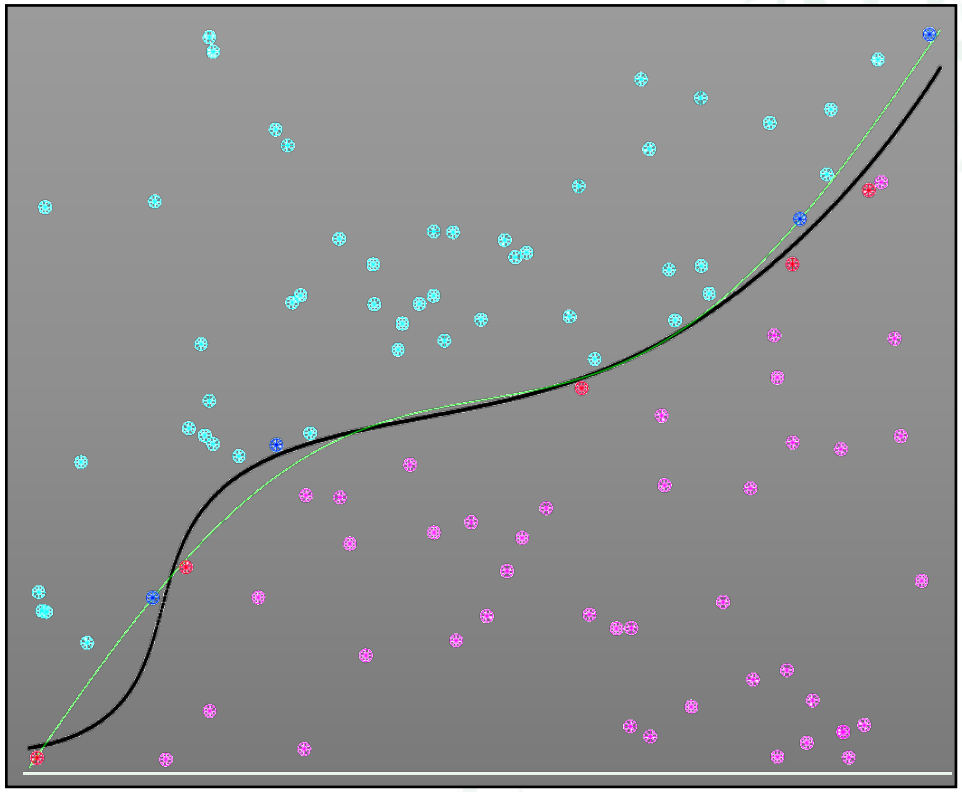
This kernel in action

Slightly non-separable case:

Transforming $\mathcal{X}$ into ∞-dimensional $\mathcal{Z}$

Overkill?

Count the support vectors!



Black line = RBF Kernel
Green line = True $f(x)$ function

Kernel formulation of SVM

Remember quadratic programming?

The only difference in the Gram matrix is now is:

$$\begin{bmatrix} y_1 y_1 \cancel{\boldsymbol{x}_1^T \boldsymbol{x}_1} K(\boldsymbol{x}_1, \boldsymbol{x}_1) & y_1 y_2 \cancel{\boldsymbol{x}_1^T \boldsymbol{x}_2} K(\boldsymbol{x}_1, \boldsymbol{x}_2) & \cdots & y_N y_1 \cancel{\boldsymbol{x}_N^T \boldsymbol{x}_1} K(\boldsymbol{x}_1, \boldsymbol{x}_N) \\ y_2 y_1 \cancel{\boldsymbol{x}_2^T \boldsymbol{x}_1} K(\boldsymbol{x}_2, \boldsymbol{x}_1) & y_2 y_2 \cancel{\boldsymbol{x}_2^T \boldsymbol{x}_2} K(\boldsymbol{x}_2, \boldsymbol{x}_2) & \cdots & y_N y_2 \cancel{\boldsymbol{x}_N^T \boldsymbol{x}_2} K(\boldsymbol{x}_2, \boldsymbol{x}_N) \\ \vdots & \vdots & \ddots & \vdots \\ y_N y_1 \cancel{\boldsymbol{x}_N^T \boldsymbol{x}_1} K(\boldsymbol{x}_N, \boldsymbol{x}_1) & y_N y_2 \cancel{\boldsymbol{x}_N^T \boldsymbol{x}_2} K(\boldsymbol{x}_N, \boldsymbol{x}_2) & \cdots & y_N y_N \cancel{\boldsymbol{x}_N^T \boldsymbol{x}_N} K(\boldsymbol{x}_N, \boldsymbol{x}_N) \end{bmatrix}$$

Everything else is the same.

Common kernel functions

Some commonly used kernel functions & their shape

- Polynomial

$$K(\boldsymbol{x}, \boldsymbol{x}') = \left(1 + \sum_d \boldsymbol{x}_d \boldsymbol{x}'_d\right)^Q$$

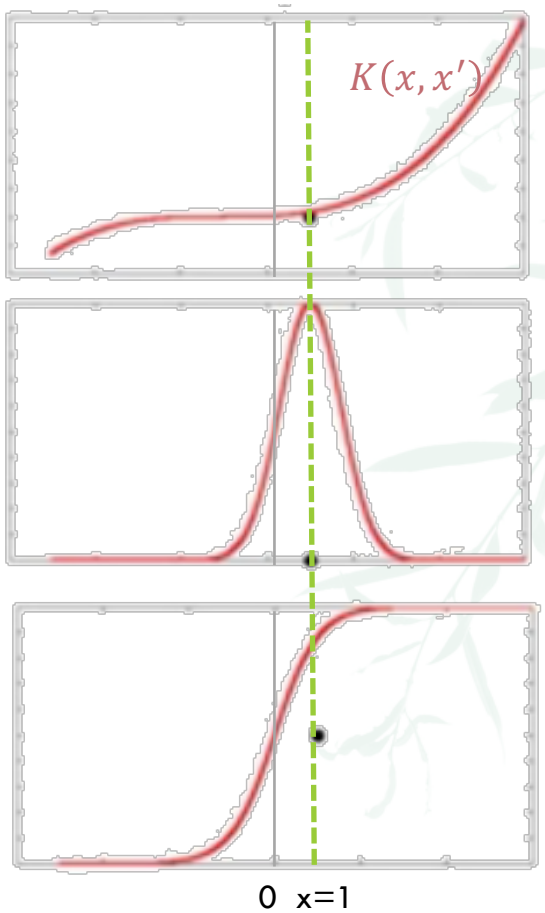
- Radial Basis Functions*

$$K(\boldsymbol{x}, \boldsymbol{x}') = \exp\left(-\frac{(\boldsymbol{x} - \boldsymbol{x}')^2}{2\sigma^2}\right)$$

*Setting $\gamma = 1/2\sigma^2$

- Saturating, sigmoid-like

$$K(\boldsymbol{x}, \boldsymbol{x}') = \tanh(c\boldsymbol{x}^T\boldsymbol{x}' + h)$$



Many for special data types:

– String similarity for text, genetics

Figure credits: Alex Ihler @ UCI

Kernels as similarity measure

$K(\boldsymbol{x}, \boldsymbol{x}')$ computes $< \Phi(\boldsymbol{x}), \Phi(\boldsymbol{x}') >$

Intuitively, $K(-,-)$ assigns high score to similar vectors; low score to (almost) orthogonal vectors

Example: Radial Basis Function $K(\boldsymbol{x}, \boldsymbol{x}') = \exp(-\frac{(\boldsymbol{x}-\boldsymbol{x}')^2}{2\sigma^2})$

If $\boldsymbol{x} \approx \boldsymbol{x}'$: $\exp(-\frac{\sim 0^2}{2\sigma^2}) \approx 1$

If $\boldsymbol{x}$ far from $\boldsymbol{x}'$: $\exp(-\frac{(\text{large number})^2}{2\sigma^2}) \approx 0$

RBF, visualised

Larger γ $(= \frac{1}{2\sigma^2})$ / smaller σ^2 yields a shaper peak in the influence of the kernel at the point $\boldsymbol{x}$ in $K(\boldsymbol{x}, \boldsymbol{x}')$.

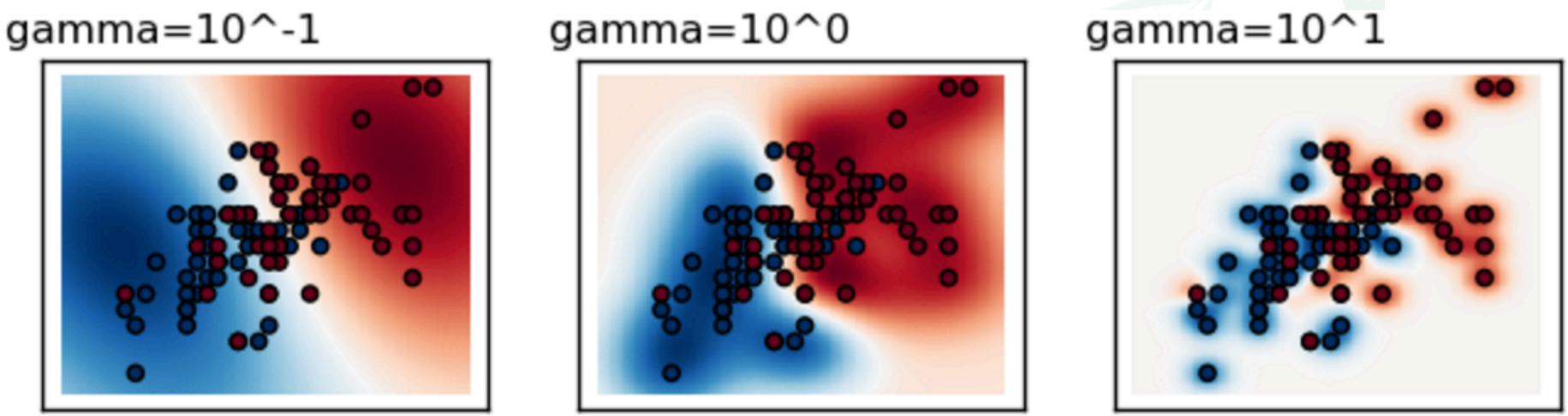
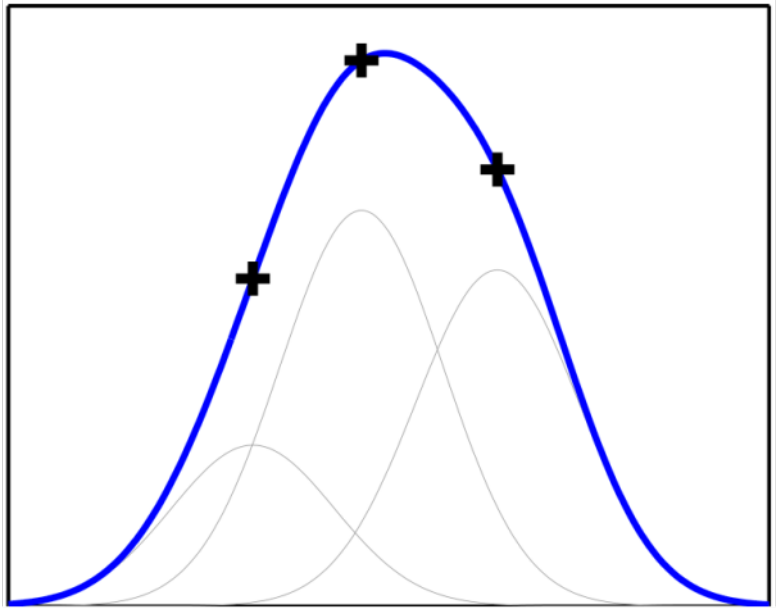
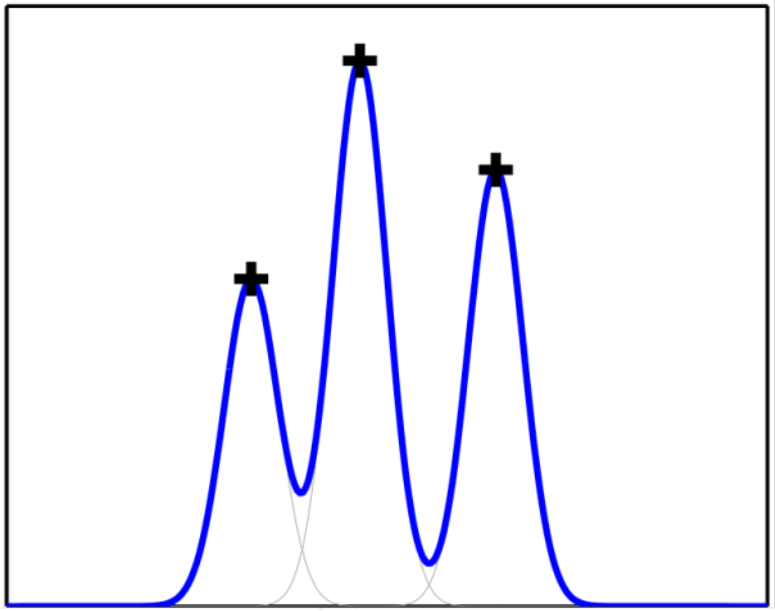


Figure credits: SciKitLearn Documentation

Quick Quiz

Which is large γ?





The final hypothesis

Express $g(\boldsymbol{x}) = \text{sign}(\boldsymbol{w}^T\boldsymbol{z} + b)$ in terms of $K(-,-)$

$$\boldsymbol{w} = \sum_{\boldsymbol{z}_n \in SV} \alpha_n y_n \boldsymbol{z}_n \Longrightarrow g(\boldsymbol{x}) = \text{sign}(\sum_{\alpha_n} \alpha_n y_n K(\boldsymbol{x}_n, \boldsymbol{x}) + b)$$

where $b = y_m - \sum_{\alpha_{n>0}} \alpha_n y_n K(\boldsymbol{x}_n, \boldsymbol{x}_m)$

for any support vector $(\alpha_m > 0)$

How do we know that $\mathcal{Z}$ exists …

… for a given $K(\boldsymbol{x}, \boldsymbol{x}')$?

Must be a valid kernel, otherwise dual form doesn't work.

Three approaches:

1. By construction
2. Mathematical properties – Mercer's Conditions
3. Who cares?

Design your own kernel

$K(x, x')$ is a valid kernel iff

1. It is symmetric, and

2. The matrix: $\begin{bmatrix} K(\boldsymbol{x}_1, \boldsymbol{x}_1) & K(\boldsymbol{x}_1, \boldsymbol{x}_2) & \dots & K(\boldsymbol{x}_1, \boldsymbol{x}_N) \\ K(\boldsymbol{x}_1, \boldsymbol{x}_1) & K(\boldsymbol{x}_2, \boldsymbol{x}_2) & \dots & K(\boldsymbol{x}_2, \boldsymbol{x}_N) \\ \vdots & \vdots & \ddots & \vdots \\ K(\boldsymbol{x}_1, \boldsymbol{x}_1) & K(\boldsymbol{x}_N, \boldsymbol{x}_2) & \dots & K(\boldsymbol{x}_N, \boldsymbol{x}_N) \end{bmatrix}$

is positive semi-definite

for any $\boldsymbol{x}_1, \dots, \boldsymbol{x}_N$ (Mercer's condition)